

PRÁCTICA 1

Sistema Bancario con Programación Orientada a Objetos en Python



PONDERACIÓN: 100
Horas Aproximadas: 10

Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas

Índice

Índice 1

Competencias 2

Objetivos 2

General 2

Específicos 2

Descripción / Enunciado 2

Entregables 3

Consideraciones 3

Cronograma 3

Evaluación 4

Valores 4

Referencias 4

Competencias

Aplicación de los conceptos de Programación Orientada a Objetos (POO) en Python, utilizando los pilares fundamentales de Abstracción, Encapsulamiento, Herencia y Polimorfismo para el desarrollo de un sistema bancario.

Objetivos

General

El objetivo de esta práctica es que los estudiantes apliquen los conceptos de Programación Orientada a Objetos (POO) en Python, utilizando los pilares fundamentales de Abstracción, Encapsulamiento, Herencia y Polimorfismo. Los estudiantes desarrollarán un sistema bancario simple que permite gestionar cuentas de ahorro y cuentas monetarias.

Específicos

- Comprender la abstracción al modelar un sistema bancario real.
- Usar herencia para crear cuentas de ahorro y monetaria.
- Crear una clase base para la cuenta bancaria con encapsulamiento.
- Aplicar polimorfismo para adaptar las operaciones a cada tipo de cuenta.

Descripción / Enunciado

Un banco necesita una aplicación que permita a sus clientes abrir y gestionar cuentas de manera sencilla. Usted como desarrollador su tarea es diseñar un sistema que modele dos tipos de cuentas: Cuenta de Ahorro y Cuenta Monetaria, cada una con características específicas.

El sistema debe contar con una clase base Cuenta, que representará una cuenta bancaria genérica, esta cuenta debe tener un titular, un número de cuenta y un saldo, además de permitir depósitos, retiros, mostrar saldo y mostrar información. El número de cuenta se generará automáticamente como un número aleatorio de 16 dígitos, asegurando que cada cuenta tenga un identificador único sin necesidad de ser ingresado manualmente por el usuario. También deberá incluir un método que facilite la actualización del saldo, garantizando que solo pueda ser modificado por las clases derivadas.

La Cuenta de Ahorro, que hereda de la clase Cuenta, incorpora una tasa de interés, permitiendo a los clientes aumentar su saldo. Para ello, se debe definir un método que calcule el interés y lo añada a la cuenta, este se obtiene multiplicando el saldo por la tasa de interés.

Por otro lado, la Cuenta Monetaria, también derivada de Cuenta, ofrecerá un límite de crédito adicional, permitiendo que los clientes puedan retirar más dinero del que poseen en su saldo, siempre dentro de un margen definido. En este caso, el método de retiro debe adaptarse para considerar este límite de crédito.

Para que el sistema sea funcional, se debe implementar un menú interactivo en consola que permita al usuario elegir entre abrir una nueva cuenta o gestionar una cuenta existente. Al abrir una cuenta, el usuario deberá seleccionar entre una cuenta de ahorro o una cuenta monetaria. Si elige gestionar una cuenta ya creada, podrá realizar depósitos, retiros, mostrar información de su cuenta y calcular interés (solo para cuentas de ahorro). El programa debe mantenerse en ejecución hasta que el usuario decida salir.

El desarrollo de esta aplicación debe seguir los principios de la Programación Orientada a Objetos (POO), asegurando que los datos estén correctamente encapsulados y que las clases hijas hereden las funcionalidades de la clase base de manera eficiente. Además, se deben aplicar conceptos de polimorfismo, permitiendo que los métodos redefinidos en cada tipo de cuenta se comporten de manera diferente según sus necesidades específicas.

Entregables

- La entrega será el día lunes 03 de marzo a más tardar a las 11:59 pm.
- La entrega será por medio de la plataforma UEDI.
- Deberá agregar al auxiliar de su sección como colaborador del repositorio.

Consideraciones

- La aplicación será mostrada en consola
- El lenguaje a utilizar será python
- Cualquier caso de copia parcial o total tendrá una nota de 0 y será reportada.

- Para dudas concernientes a la práctica se utilizarán los foros en UEDI de manera que todos los estudiantes puedan ver las preguntas y las posteriores respuestas.

Cronograma

Tarea	Fecha
Asignación de la práctica / Entrega del enunciado	Febrero 2025
Fecha de entrega	03 de marzo de 2025 (11:59 pm)
Fecha de calificación	Por definir

Evaluación

La evaluación de la práctica busca medir el cumplimiento de los objetivos planteados, así como la correcta aplicación de los conocimientos técnicos y habilidades.

Criterio	Descripción	Puntos Máximos
Implementación de Abstracción	Creación de clase abstracta y aplicación correcta del concepto	20
Implementación de Herencia	Correcta implementación de herencia en atributos y métodos	20
Implementación de Encapsulamiento	Uso adecuado de atributos privados, getters y setters	25
Implementación de Polimorfismo	Sobrescritura de métodos para diferentes comportamientos	10
Funcionalidad del Sistema	Correcta implementación de todas las funcionalidades requeridas	25

Valores

En el desarrollo de la práctica, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

Originalidad del Trabajo

Cada estudiante o equipo debe desarrollar su propio código y/o documentación, aplicando los conocimientos adquiridos en el curso.

Prohibición de Copias y Plagio

Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de 0 puntos.

Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.

Uso Responsable de Recursos Externos

El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencian correctamente y se comprendan plenamente.

Revisión y Detección de Plagio

Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.

En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual o en equipo. Si este extremo no es comprobable la calificación será de 0 puntos.

Al detectarse estos aspectos se informará al catedrático del curso quien realizará las acciones que considere oportunas.

Referencias

Python Documentation: <https://docs.python.org/3/>

Object-Oriented Programming in Python: <https://realpython.com/python3-object-oriented-programming/>

Python OOP Tutorials: <https://www.programiz.com/python-programming/object-oriented-programming>